Machine Learning Operations (AIMLCZG523)

Assignment 1

Heart Disease Risk Prediction:

End-to-End MLOps from Raw Data to a Monitored Kubernetes API

Student: Sai Rakshith

BITS ID: 2025AE05180

Programme: M.Tech AI/ML, BITS Pilani (WILP)

Code repository: https://github.com/Sai-Rakshith-2209/heart-mlops

Date: 20 October 2026

1. Summary and architecture

A binary classifier predicts presence of heart disease from 13 clinical attributes of the UCI Cleveland dataset and is served as a hardened, monitored REST API. The champion model is **Logistic Regression (C=1, l2)**, selected on 5-fold cross-validated ROC-AUC (0.907). On the untouched 20% test set it reaches ROC-AUC 0.960, accuracy 0.869 and recall 0.929.

Every stage is code in a single Python package (src/heart) that is executed identically by a developer, the notebook and the CI runner, so there is one implementation of each step. The figure below shows the end-to-end flow.

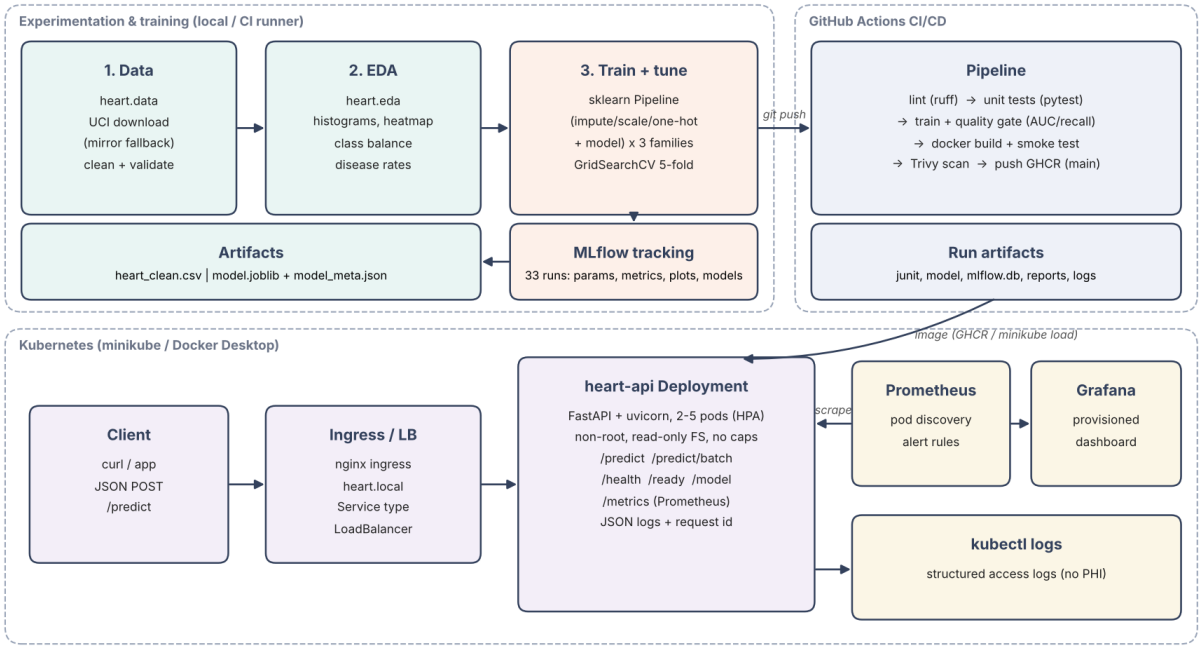


Figure 1. System architecture: training and tracking, CI/CD, Kubernetes serving and monitoring

Assignment task	Where it is implemented
1. Data acquisition and EDA	src/heart/data.py, src/heart/eda.py, notebooks/
2. Features and models	src/heart/features.py, src/heart/train.py
3. Experiment tracking	MLflow (sqlite backend) inside train.py
4. Packaging and reproducibility	models/model.joblib + model_meta.json, MLflow model, pinned requirements
5. CI/CD and tests	.github/workflows/ci.yml, tests/ (21 pytest tests)
6. Containerisation	Dockerfile, scripts/docker_smoke_test.sh
7. Deployment	k8s/*.yaml, scripts/ci_deploy.sh (CI deploy job)
8. Monitoring and logging	/metrics, JSON logs, k8s/monitoring (Prometheus + Grafana)

2. Setup and installation

Requirements: Python 3.10-3.12 (3.12 on the Prayogshala VM, 3.11 on the Windows laptop, in CI and in the container; 3.10 verified separately). Docker and Kubernetes run in the CI pipeline (section 7), so no local container runtime is required. On Prayogshala:

```
cd ~/workspace && git clone https://github.com/Sai-Rakshith-2209/heart-mlops.git && cd heart-mlops
python3 -m venv .venv && source .venv/bin/activate     # Windows Git Bash: py -3.11 -m venv .venv &&
source .venv/Scripts/activate
pip install -r requirements-dev.txt -e .
python -m heart.data     # download + clean  -> data/processed/heart_clean.csv
python -m heart.eda      # EDA figures       -> reports/figures/
```

```
python -m heart.train    # tune + track      -> mlflow.db, models/
pytest -v && ruff check .
make mlflow              # MLflow UI  http://127.0.0.1:5000
uvicorn heart.api:app --port 8000       # API, interactive docs at /docs
./scripts/docker_smoke_test.sh          # Task 6 (any host with Docker)
./scripts/ci_deploy.sh                  # Tasks 7-8 (any Linux host with Docker + minikube; runs in
CI)
```

Dependencies are split into three pinned files: `requirements-serve.txt` (the slim runtime baked into the image), `requirements.txt` (adds MLflow, plotting, skops for training) and `requirements-dev.txt` (adds pytest, httpx, ruff). All pins were verified by clean installs on Python 3.10, 3.11 and 3.12 (identical metrics on all three) and resolve to binary wheels on Linux and Windows.

3. Data acquisition and EDA

3.1 Acquisition and cleaning

heart.data downloads the official UCI file `processed.cleveland.data`, and falls back to a GitHub mirror of the same 303 rows (string-coded categoricals mapped back to the UCI codebook) if the UCI host is unreachable. Cleaning then:

- coerces every column to numeric and treats '?' as missing;
- nulls out values outside the UCI codebook domains (e.g. cp not in 1-4) or physiologically impossible ranges;
- drops exact duplicate rows;
- binarises the target: `num = 0` means no disease, `num 1-4` means disease present.

Result: 303 rows, 6 missing cells (ca: 4, thal: 2). Missing values are deliberately not imputed here: imputation is part of the model pipeline so its statistics are learned on training folds only, preventing leakage, and the same logic runs at inference.

3.2 Exploratory analysis

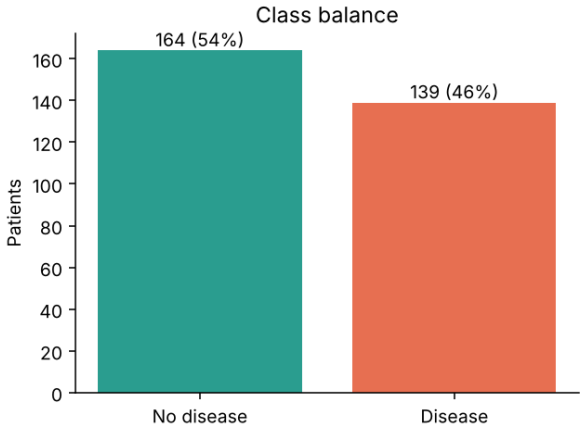


Figure 2. Class balance: 45.9% positive

The classes are close to balanced, so no resampling is needed; stratified splits keep the ratio in every fold. ROC-AUC and recall are still reported as the primary metrics because missing a sick patient is costlier than a false alarm.

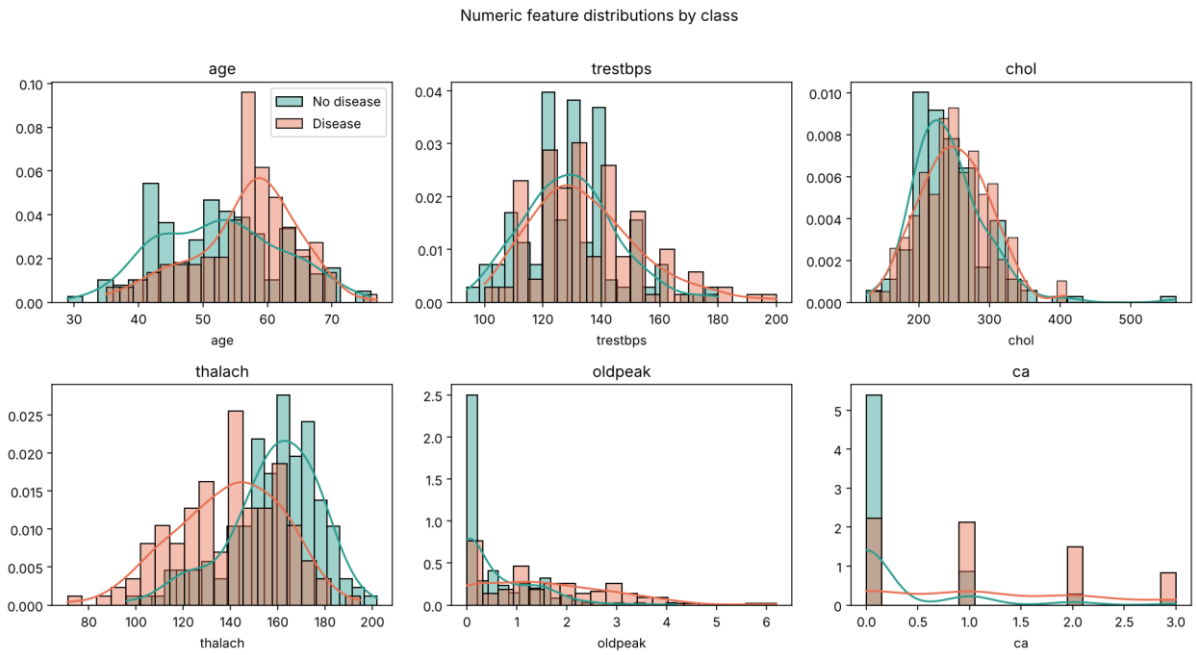


Figure 3. Numeric feature distributions by class

Patients with disease show markedly lower maximum heart rate (thalach), higher ST depression (oldpeak) and more fluoroscopy-coloured vessels (ca). Cholesterol and resting blood pressure distributions largely overlap between classes.

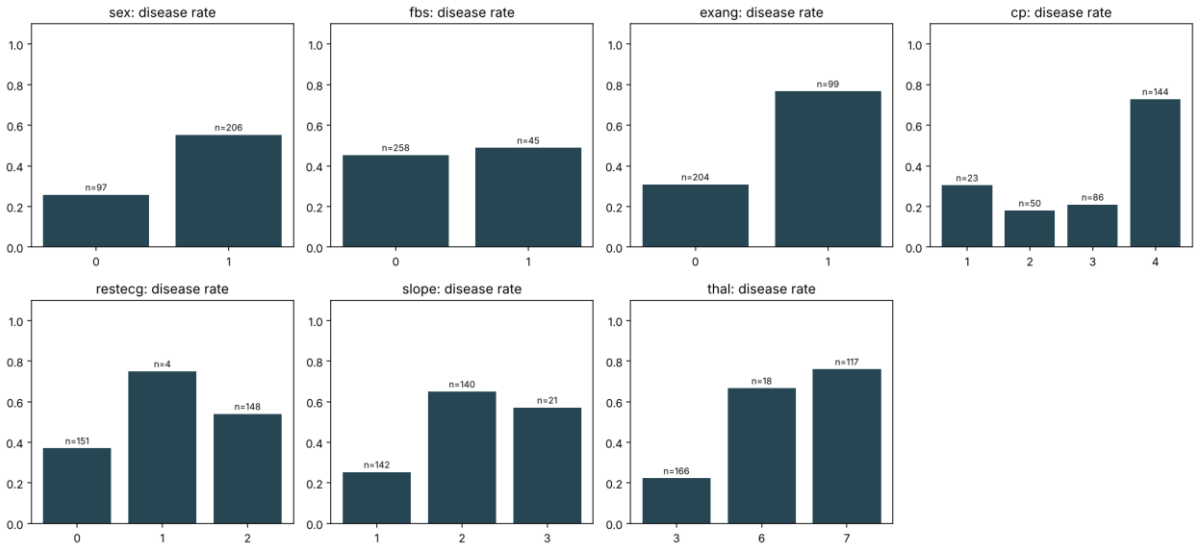


Figure 4. Disease rate per category (n per bar)

Asymptomatic chest pain (cp = 4), exercise-induced angina (exang = 1) and reversible thallium defects (thal = 7) each carry disease rates above 70%. restecg = 1 has only 4 patients, which is why categorical features are one-hot encoded with handle_unknown='ignore' rather than relying on rare levels.

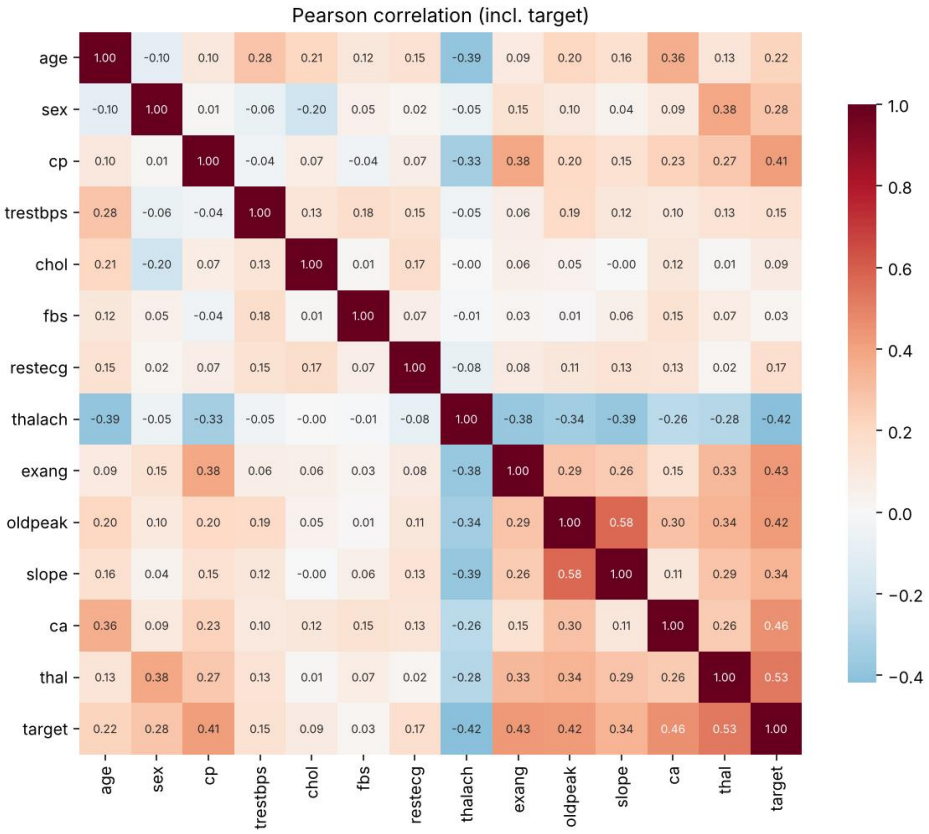


Figure 5. Pearson correlation matrix including the target

Feature	Correlation with target	Feature	Correlation with target
thal	0.526	sex	0.277
ca	0.460	age	0.223
exang	0.432	restecg	0.169
oldpeak	0.425	trestbps	0.151
thalach	-0.417	chol	0.085
cp	0.414	fbs	0.025
slope	0.339		

Feature correlations are moderate (none above |0.6|), so multicollinearity is not a concern and all 13 features are retained. fbs and chol carry little linear signal; dropping both moves CV ROC-AUC from 0.907 to 0.911, a +0.004 change well inside one CV standard deviation (about 0.02). They are therefore kept: the gain is noise-level, and L2 regularisation already shrinks weak coefficients.

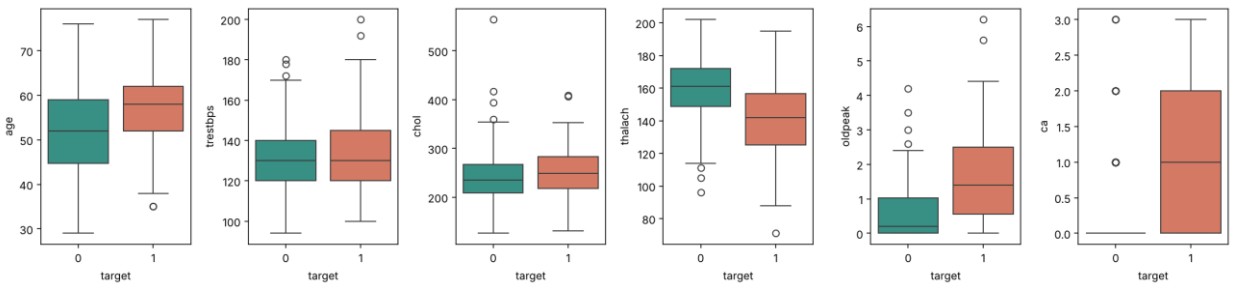


Figure 6. Box plots of numeric features by class (outlier check)

A few high cholesterol values (up to 564 mg/dl) are extreme but clinically plausible, so they are kept rather than clipped; StandardScaler and tree models are robust enough at this scale.

4. Feature engineering and model development

4.1 Preprocessing pipeline

Group	Columns	Transform
Numeric	age, trestbps, chol, thalach, oldpeak, ca	median imputation, then StandardScaler
Binary	sex, fbs, exang	most-frequent imputation (already 0/1)
Categorical	cp, restecg, slope, thal	most-frequent imputation, then one-hot (unknown -> all zeros)

The ColumnTransformer and the estimator are wrapped in a single sklearn **Pipeline**, producing 22 model inputs. Because the whole fitted Pipeline is the deployable artifact, preprocessing at inference is exactly the training code: there is no train/serve skew.

4.2 Model selection and tuning

Data was split 80/20 with stratification (seed 42): 242 training and 61 test rows. Three model families were tuned with GridSearchCV over 5-fold stratified CV, scoring accuracy, precision, recall, F1 and ROC-AUC on every fold, and refitting on ROC-AUC. The test set is used once, for reporting only; the champion is selected on CV ROC-AUC.

Model	Search space	Best parameters
Logistic Regression (liblinear)	C in {0.01, 0.1, 0.3, 1, 3} x penalty in {l1, l2} (10)	C = 1.0, l2
Random Forest	n_estimators {200, 400} x max_depth {None, 4, 8} x min_samples_leaf {1, 5} (12)	200 trees, max_depth 4, min_samples_leaf 5
HistGradientBoosting	learning_rate {0.03, 0.1} x max_depth {2, 3} x max_iter {100, 200} (8)	lr 0.03, max_depth 2, 100 iterations

4.3 Results

Model	CV ROC-AUC	CV recall	Test ROC-AUC	Test acc.	Test prec.	Test recall	Test F1
Logistic Regression	**0.907**	**0.774**	**0.960**	**0.869**	**0.813**	**0.929**	**0.867**
Random Forest	0.901	0.774	0.955	0.885	0.839	0.929	0.881
HistGradientBoosting	0.889	0.783	0.951	0.885	0.862	0.893	0.877

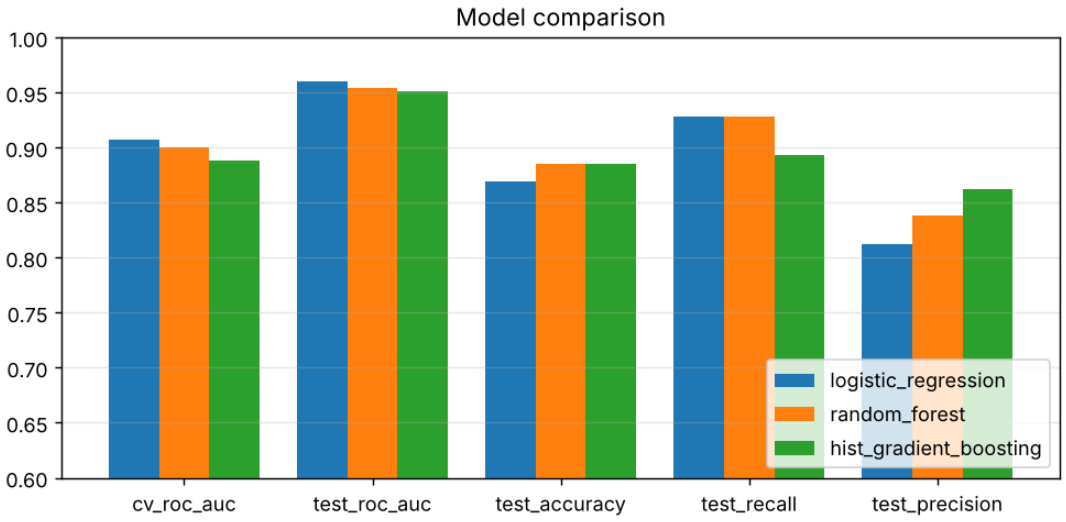


Figure 7. CV and test metrics per model family

Why Logistic Regression wins. All three families are within about 0.02 CV ROC-AUC, which is inside one CV standard deviation (about 0.02). Given equal performance, the linear model is the better production choice: it has the highest CV ROC-AUC, the lowest CV variance, interpretable coefficients for clinicians, a sub-millisecond inference cost and the smallest artifact. The tuned tree models also chose their most constrained settings (shallow depth, larger leaves), a sign that the dataset (303 rows) rewards simple models.

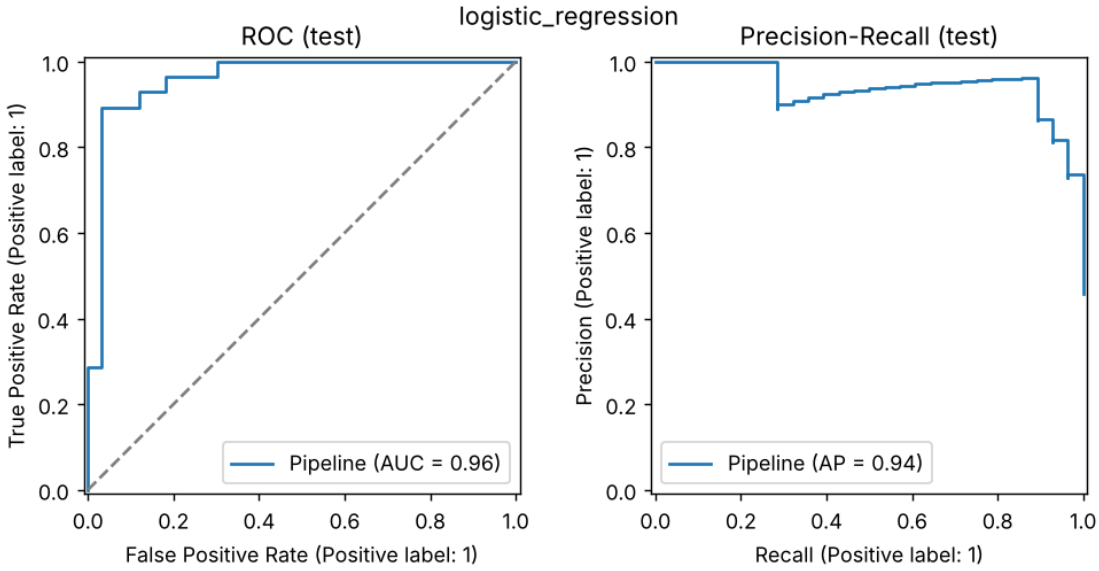


Figure 8. Champion ROC and precision-recall curves on the test set

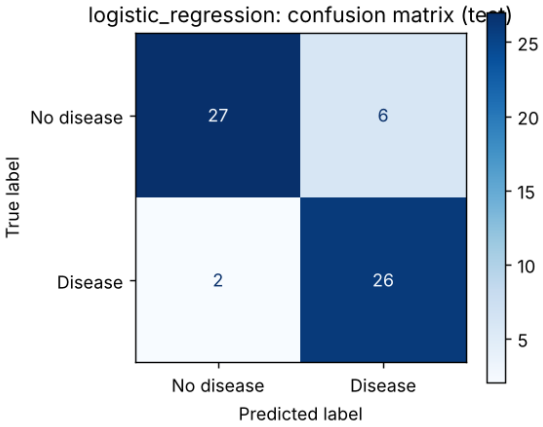


Figure 9. Champion confusion matrix (threshold 0.5)

At the default threshold the champion misses 2 of 28 diseased test patients (recall 0.93) at the cost of 6 false positives. The threshold is stored in model_meta.json and read by the API, so it can be tuned toward recall without code changes.

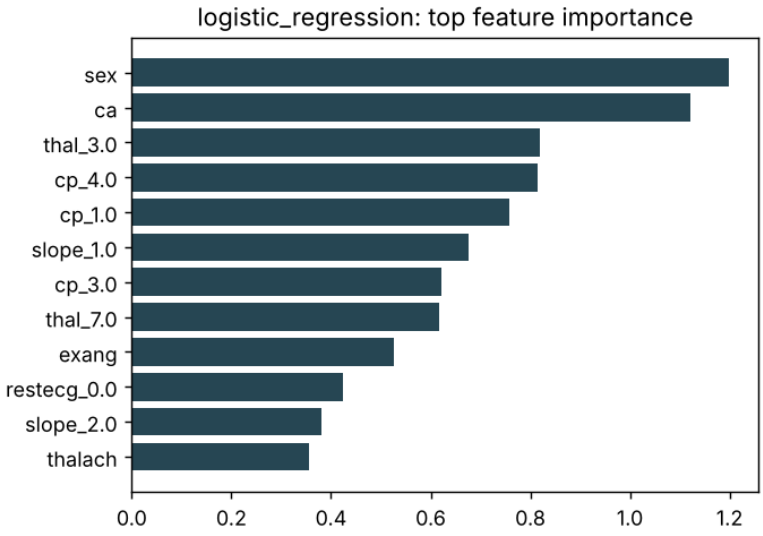


Figure 10. Champion: absolute standardised coefficients

5. Experiment tracking (MLflow)

Tracking uses an MLflow sqlite backend (mlflow.db) with local artifact storage. Each model family is a parent run; each grid candidate is a nested child run, so every configuration evaluated is recorded:

Level	Count	Logged
Parent run per family	3	split sizes, seed, CV folds, best params; CV + test metrics; confusion matrix, ROC/PR and importance plots; MLflow model with signature and input example; tags: data SHA-256, git SHA
Nested candidate runs	30	hyperparameters; mean and std of every CV metric; train ROC-AUC (overfitting check)
Champion tag	1	champion=true, model_meta.json, model comparison plot

MLflow 3 serialises scikit-learn models with **skops** instead of pickle. The training code allow-lists exactly the types present in each fitted model, computed from the object itself, rather than trusting arbitrary types.

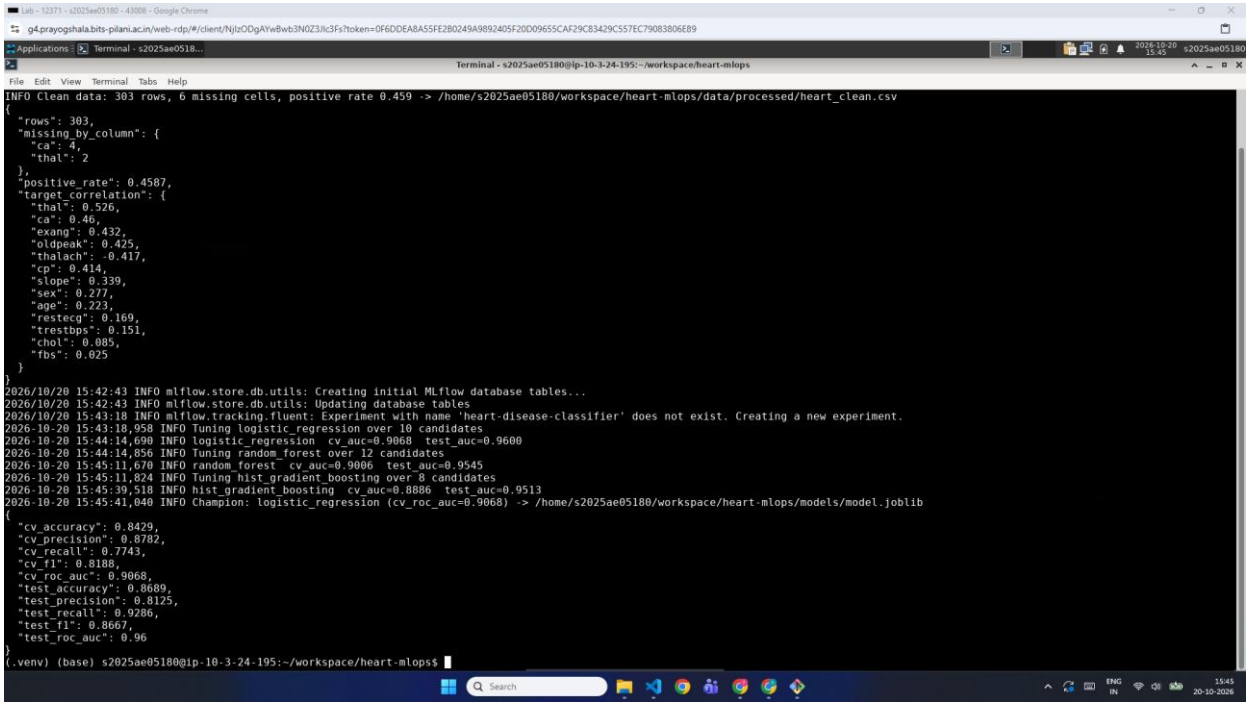


Figure 11. Data, EDA and training on the Prayogshala VM: identical data profile and metrics (CV ROC-AUC 0.9068, test ROC-AUC 0.9600)

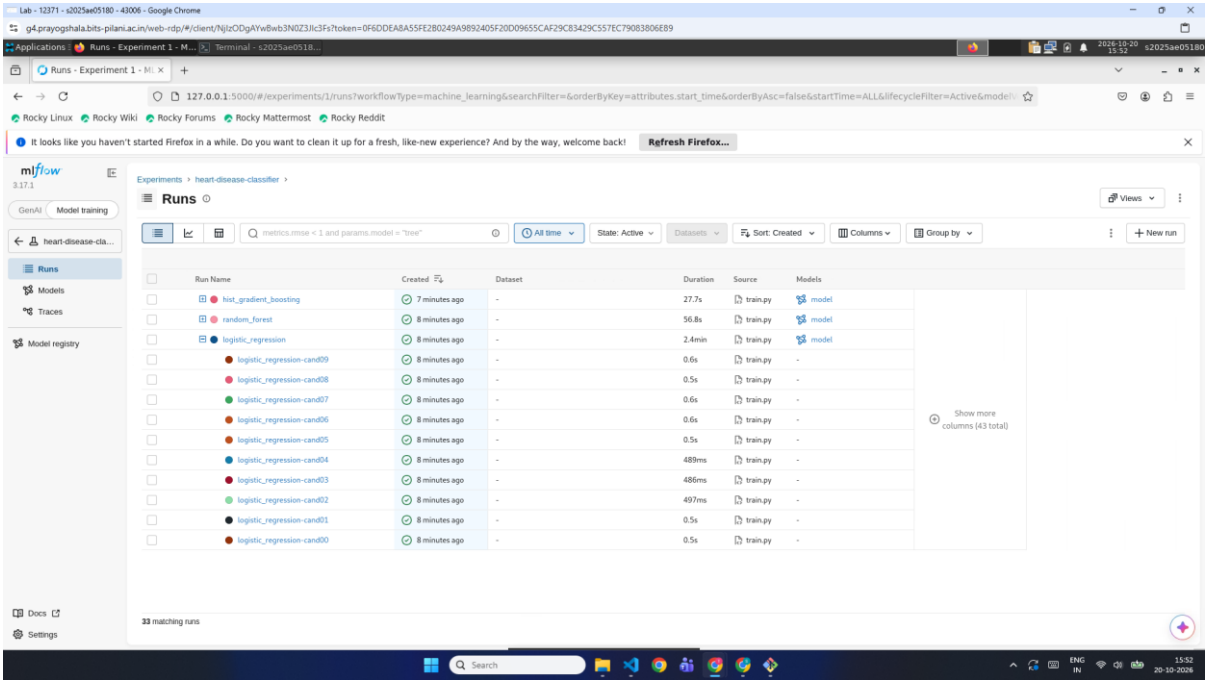


Figure 12. MLflow experiment heart-disease-classifier: parent run per model family with nested candidate runs (Prayogshala)

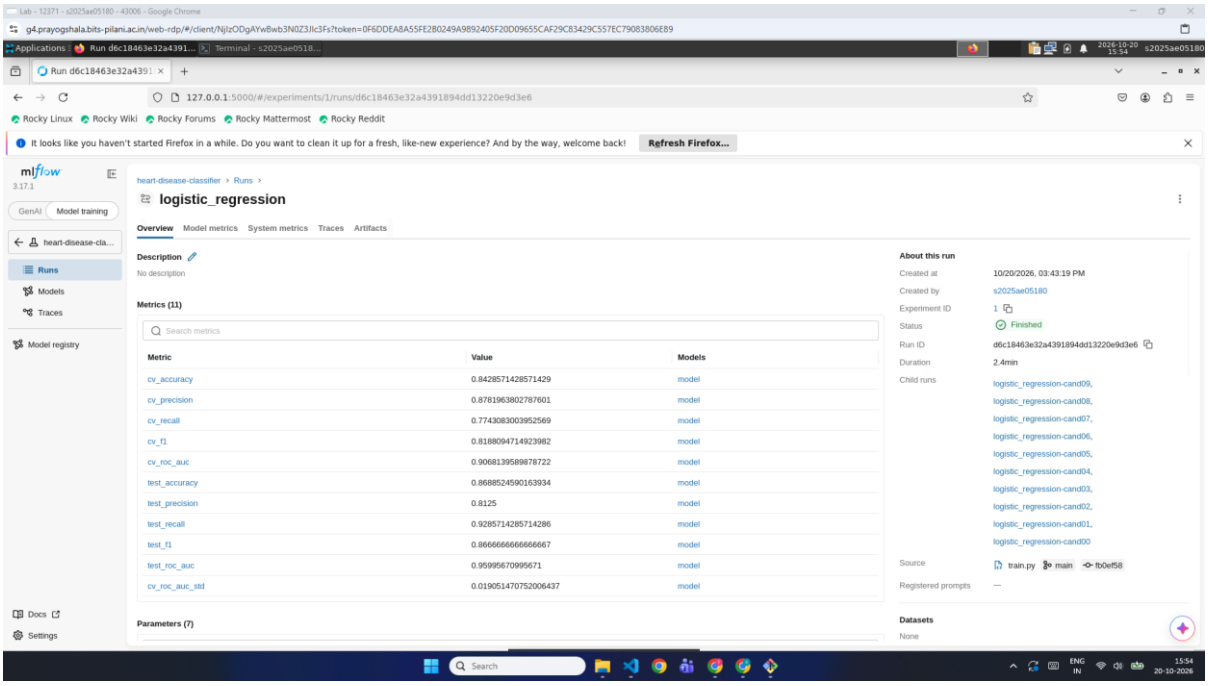


Figure 13. Champion run on Prayogshala: CV and test metrics, its 10 child runs, and source traceability (train.py, branch main, commit fb0ef58)

6. Model packaging and reproducibility

- **Artifact:** `models/model.joblib` is the full fitted Pipeline (preprocessing + model). The same model is also logged as an MLflow model.
- **Metadata:** `model_meta.json` records the MLflow run id (4ea167ff7e52...), metrics, decision threshold, feature order, training-data SHA-256, timestamp and the Python and scikit-learn versions.
- **Determinism:** fixed random_state for the split, CV shuffling and every estimator; a unit test asserts identical predictions from two independent fits.
- **Pinned environment:** exact versions in the three requirements files; numpy, pandas and scikit-learn are identical between training and serving because the artifact is a fitted sklearn object.

- **Data lineage:** the cleaned CSV is committed and its hash is stored with every run; heart.data reproduces it from the source.

7. CI/CD pipeline and automated testing

7.1 Unit tests (21, pytest)

File	What is tested
test_data.py (5)	UCI parsing marks '?' missing; mirror codes map to the UCI codebook; target binarisation; out-of-domain values nulled and duplicates dropped; cleaned-data contract (303 rows, binary target)
test_model.py (7)	preprocessor output width (22) and finiteness; tolerance of missing and unseen categories; CV ROC-AUC above 0.85; determinism; each model family trains and predicts valid probabilities
test_api.py (9)	/health and /ready; /predict returns prediction and confidence; optional fields; 422 for out-of-range, invalid code and unknown field; batch and 413 limit; /metrics content; metric labels stay bounded for unknown URLs; x-request-id validation

API tests train a small model on the fly, so the test job does not depend on the training job's artifacts.

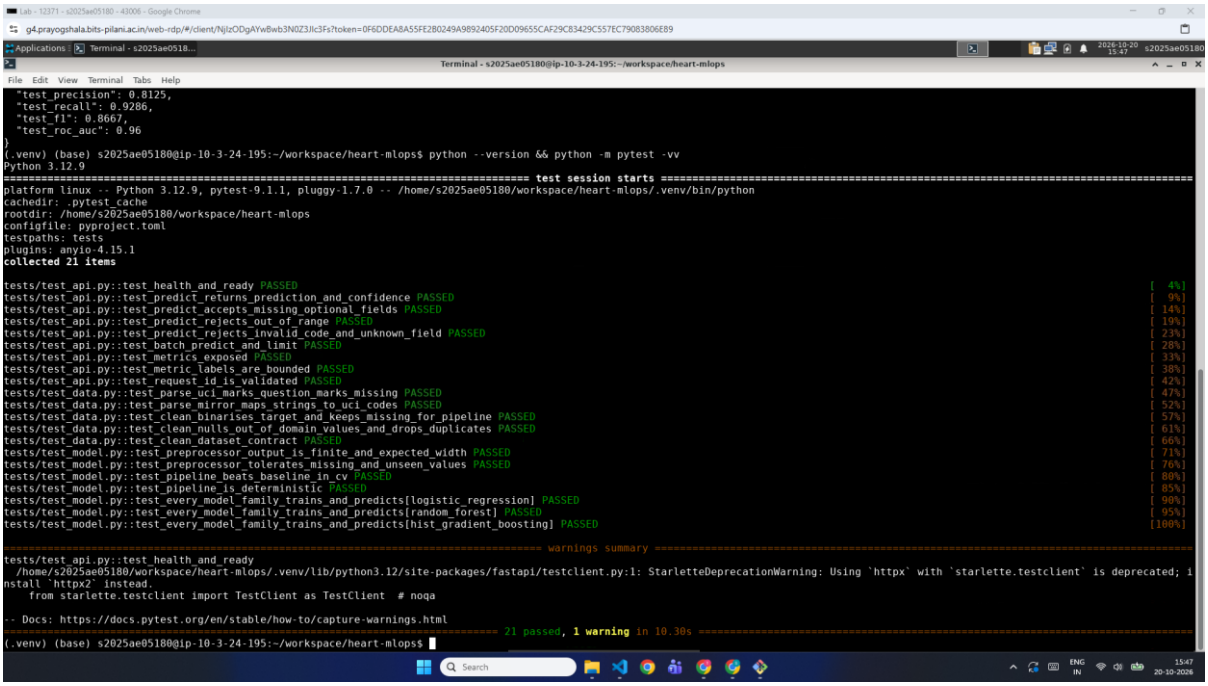


Figure 14. Full test suite on the Prayogshala VM (Python 3.12.9): 21 passed

7.2 GitHub Actions workflow

Job	Steps	Fails the build when
lint	ruff check (pyflakes, pycodestyle, isort, bugbear, pyupgrade), ruff format --check	any lint or formatting violation
test	pytest -v with JUnit XML uploaded as an artifact	any test fails
train (needs lint, test)	download + clean, check the data is byte-identical to the committed copy, EDA, tune + track, quality gate; uploads model, mlflow.db, reports, train log	data changed, or CV/test ROC-AUC below 0.85, or test recall below 0.80
docker (needs train)	build the image with this run's model, run it, assert /ready, /predict contract, 422 on bad input, healthy status, non-root user; push to GHCR on main	container does not serve correctly
deploy (needs docker)	start minikube, deploy API + Prometheus + Grafana, call /predict via LoadBalancer and Ingress, generate traffic, capture screenshots, logs and Prometheus state	any rollout, endpoint or evidence step fails

Every third-party action is pinned to a full commit SHA rather than a tag. Tags can be moved by whoever controls the action repository, which is exactly how the 2026 trivy-action compromise worked; a SHA cannot be moved. Checkout does not persist credentials, the workflow token is read-only except for package write in the push step, superseded runs are cancelled, and pip and Docker layers are cached.

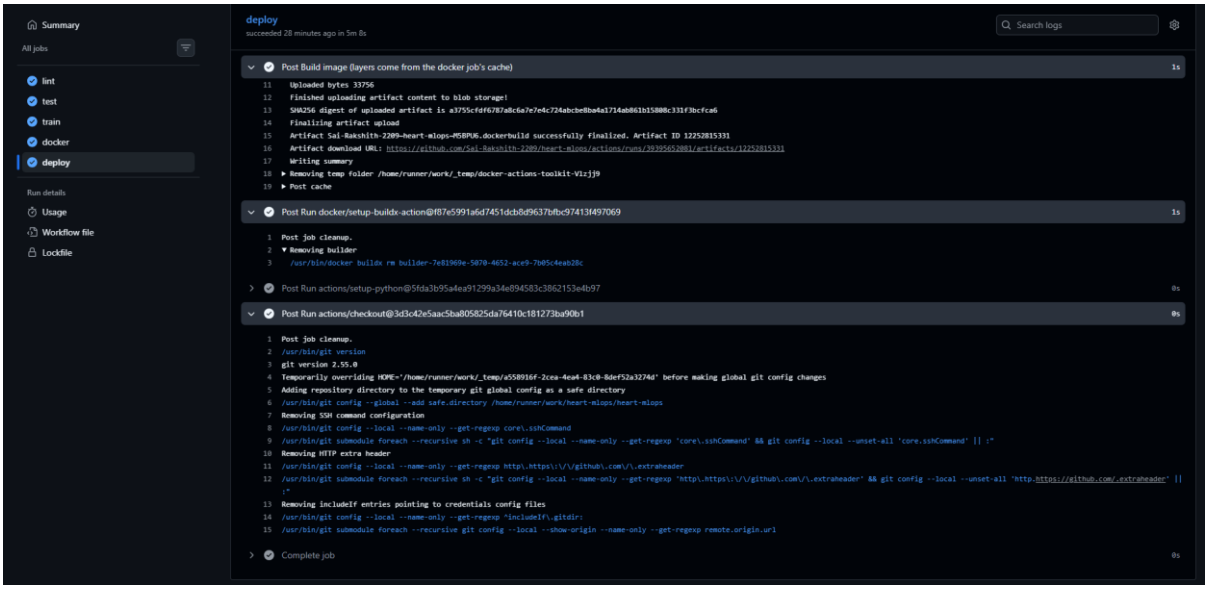


Figure 15. GitHub Actions run: lint, test, train, docker and deploy all green



Figure 16. Artifacts produced by every run: test results, model, MLflow store and reports, smoke-test output, deployment evidence

8. Model containerisation

The Dockerfile is multi-stage: a deps stage builds a virtualenv from requirements-serve.txt only; the runtime stage copies the venv, three source files and the model. MLflow, plotting libraries and compilers are not in the image. .dockerignore is an allow-list, so nothing else (data, tests, mlflow.db) can leak into the build context.

Control	Implementation
Least privilege	fixed non-root UID 10001 (verifiable by runAsNonRoot)
Health	HEALTHCHECK calls /ready (model loaded), not just process liveness
Contract	POST /predict accepts JSON, returns prediction, label, confidence, probability_disease, model name and version
Traceability	model_meta.json inside the image; served on /model and as the model_info metric

Verified output from the API serving the champion (the same files the image contains, in a clean environment with only requirements-serve.txt):

```
$ curl -X POST localhost:8000/predict -H 'Content-Type: application/json' -d @sample_request.json
{"prediction":1,"label":"heart_disease","confidence":0.8986,"probability_disease":0.8986,
 "model_name":"logistic_regression","model_version":"1.0.0"}
```

```
$ curl -X POST localhost:8000/predict -d '{"age":41,"sex":0,"cp":2, ... "thal":3}'
{"prediction":0,"label":"no_heart_disease","confidence":0.9573,"probability_disease":0.0427, ...}
$ curl -X POST localhost:8000/predict -d '{"age":500}'   ->  HTTP 422
```



Figure 17. Container smoke test in CI: /ready, /predict, 422 on invalid input, healthy HEALTHCHECK, non-root uid 10001

9. Production deployment (Kubernetes)

Manifest	Purpose and key settings
00-namespace.yaml	heart and monitoring namespaces enforcing the restricted Pod Security Standard
10-deployment.yaml	2 replicas, RollingUpdate maxUnavailable 0; startup, readiness (/ready) and liveness (/health) probes; requests and memory limits; read-only root FS, all capabilities dropped, seccomp RuntimeDefault, no service-account token; topology spread across nodes
20-service.yaml	Service type LoadBalancer on port 80
30-ingress.yaml	optional nginx Ingress (heart.local) with per-client rate limit and body-size cap, for clusters with an ingress controller; on Docker Desktop the API is exposed through the LoadBalancer
40-hpa-pdb.yaml	HPA 2-5 replicas at 70% CPU; PodDisruptionBudget minAvailable 1
50-prometheus-rbac.yaml	namespaced Role letting Prometheus list pods in heart only

All manifests pass strict schema validation (kubeconform: 17 resources valid). The deployment runs on a local minikube cluster created inside the CI runner by scripts/ci_deploy.sh, so every push redeploys from scratch and proves the manifests work: the image built in the docker job is loaded into minikube, the API is exposed through a LoadBalancer Service (minikube tunnel assigns the external IP) and through the nginx Ingress, and the same script runs unchanged on any Linux host with Docker and minikube. Docker Desktop's Kubernetes is supported through scripts/deploy_docker_desktop.sh.

```
minikube start --driver=docker && minikube addons enable ingress metrics-server
minikube image load heart-disease-api:1.0.0
kubectl apply -f k8s/ && kubectl apply -k k8s/monitoring
minikube tunnel &                          # LoadBalancer EXTERNAL-IP
curl -X POST http://<EXTERNAL-IP>/predict -H 'Content-Type: application/json' -d
@sample_request.json
curl -X POST -H 'Host: heart.local' http://$(minikube ip)/predict ...     # via Ingress
```

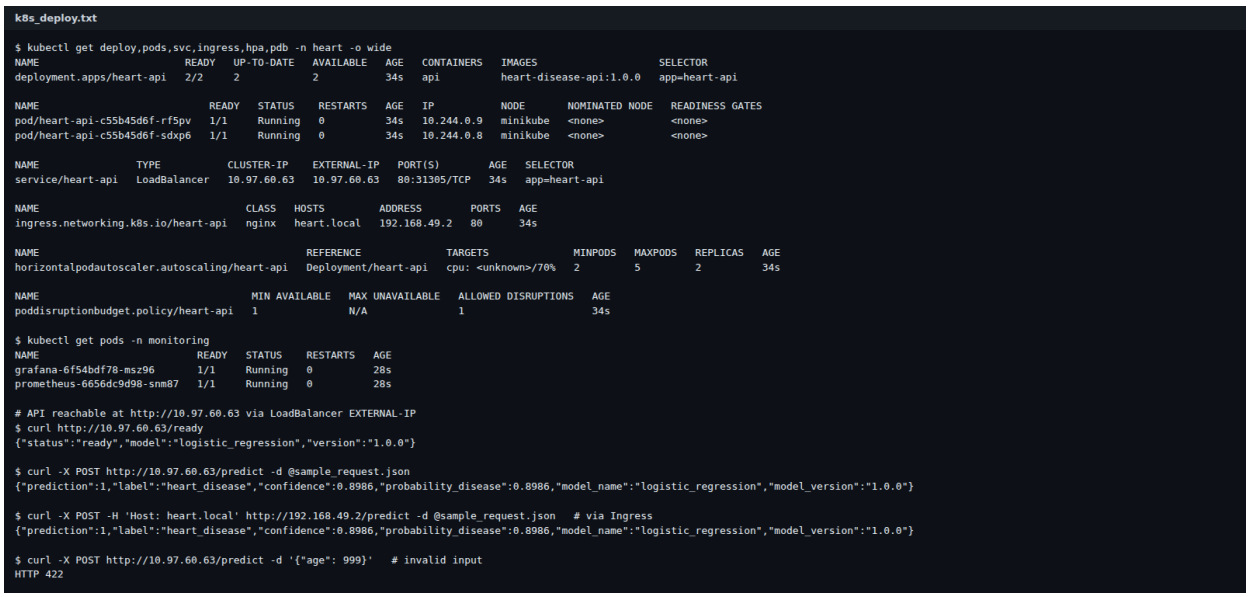


Figure 18. Kubernetes resources (2/2 pods, LoadBalancer, Ingress, HPA, PDB) and predictions via LoadBalancer and Ingress

10. Monitoring and logging

Logging. Every request emits one JSON line with request id (propagated from x-request-id or generated, and echoed in the response header), method, matched route, status, latency and client. Predictions log only the label and probability. Patient features are never written to logs, since this is health data.

```
{"ts":"2026-10-19T20:33:03+0000","level":"INFO","logger":"heart.api","msg":"request",
 "request_id":"sai-test-001","method":"POST","route":"/predict","status":200,"latency_ms":53.41}
```

Metrics. /metrics exposes http_requests_total (route, status), http_request_duration_seconds histogram, model_predictions_total (label), model_disease_probability histogram and model_info. Prometheus discovers every pod through annotations, so each replica is scraped individually rather than through the load-balanced Service. Metric labels use the matched route template, never the raw URL: unknown paths collapse into a single "unmatched" label, so a scanner cannot create unbounded time series (a cardinality-explosion denial of service).

Grafana panel	Query intent
Requests/s, p95 latency, 5xx error ratio	service health (golden signals)
Request rate by route and status	traffic mix and 4xx validation failures
Latency p50/p95/p99 for /predict	performance regression
Predictions by class; disease-positive share	output distribution
Mean predicted P(disease) over 15 min	simple drift signal: a sustained shift means input data changed

Alert rules fire when no replica is scraped for 1 minute, when more than 5% of requests fail for 5 minutes, or when p95 /predict latency exceeds 500 ms for 10 minutes. The Grafana admin password is generated randomly into a Secret; Grafana and Prometheus are reachable only through kubectl port-forward.

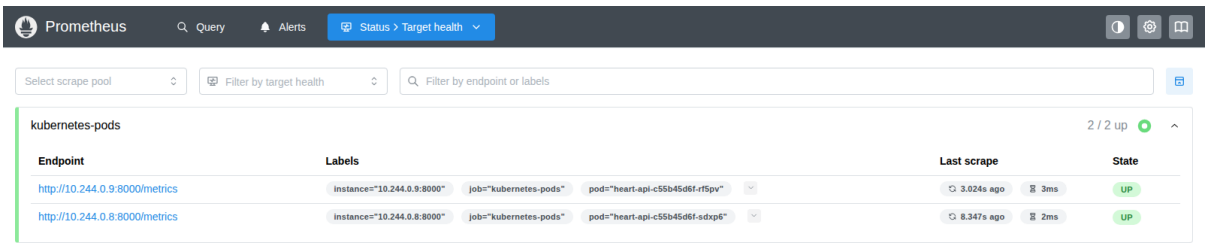


Figure 19. Prometheus target health: every heart-api pod discovered and UP

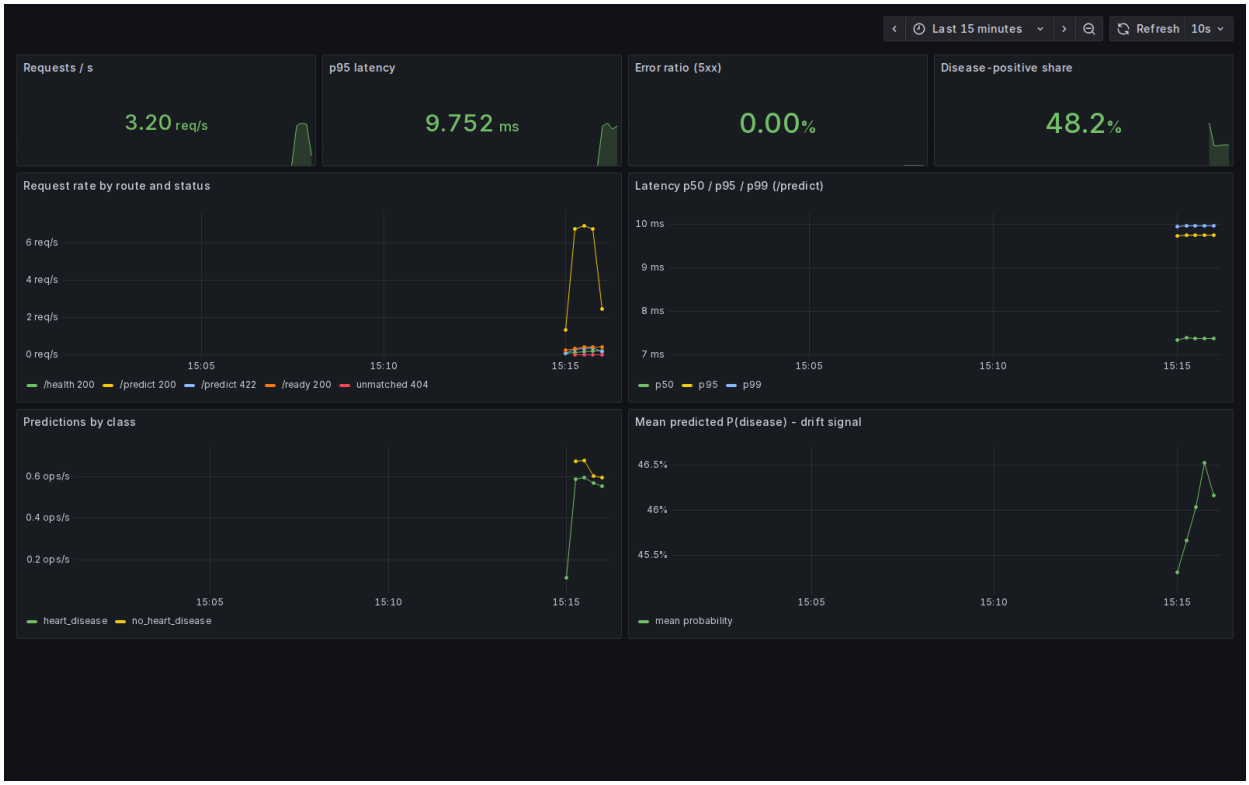


Figure 20. Grafana 'Heart Disease API' dashboard after the load generator (400 requests, 5% deliberately invalid)



pod_logs.txt

```
$ kubectl -n heart logs deploy/heart-api --tail=15
{"ts": "2026-10-20T09:45:13+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "3647b66d4bdb4aac984bf57f04760e82", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.67, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:14+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "e16332fc287c4069abaf2cb69c2e18ca", "method": "GET", "route": "/health", "status": 200, "latency_ms": 0.56, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:18+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "7ba87d257375407fbe3c859e4562ab74", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.58, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:23+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "43a0ae09021348f48643180e8a03c110", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.59, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:24+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "5362b03a08dd46c48e6db946c320f2ec", "method": "GET", "route": "/health", "status": 200, "latency_ms": 0.59, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:28+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "d4d4b9776e9646429012e106c48a97b4", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.54, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:33+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "94af5d3fa8a44648b108f47a5421cb45", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.6, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:34+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "1a7d796f5e324627ae9272074333d7dc", "method": "GET", "route": "/health", "status": 200, "latency_ms": 0.56, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:38+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "62dc33357e8f4b06a667f43da9aefe9b", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.57, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:43+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "976fa472ab144133b3962ff5cb215987", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.65, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:44+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "179f3c91a53149589190532ec2d6a1be", "method": "GET", "route": "/health", "status": 200, "latency_ms": 0.56, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:48+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "5ba2775d515847f9ab9284a1d6c2ca96b", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.61, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:53+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "ecca4537f34847a68e48a5a4f6f5354a", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.53, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:54+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "87a39df77a7245477b76222e14d89b8bb", "method": "GET", "route": "/health", "status": 200, "latency_ms": 0.6, "client": "10.244.0.1"}
{"ts": "2026-10-20T09:45:58+0000", "level": "INFO", "logger": "heart.api", "msg": "request", "request_id": "92872ac366a847e9b13b4212763b26e4", "method": "GET", "route": "/ready", "status": 200, "latency_ms": 0.7, "client": "10.244.0.1"}
```

Figure 21. Structured JSON request logs from a pod (no patient features logged)

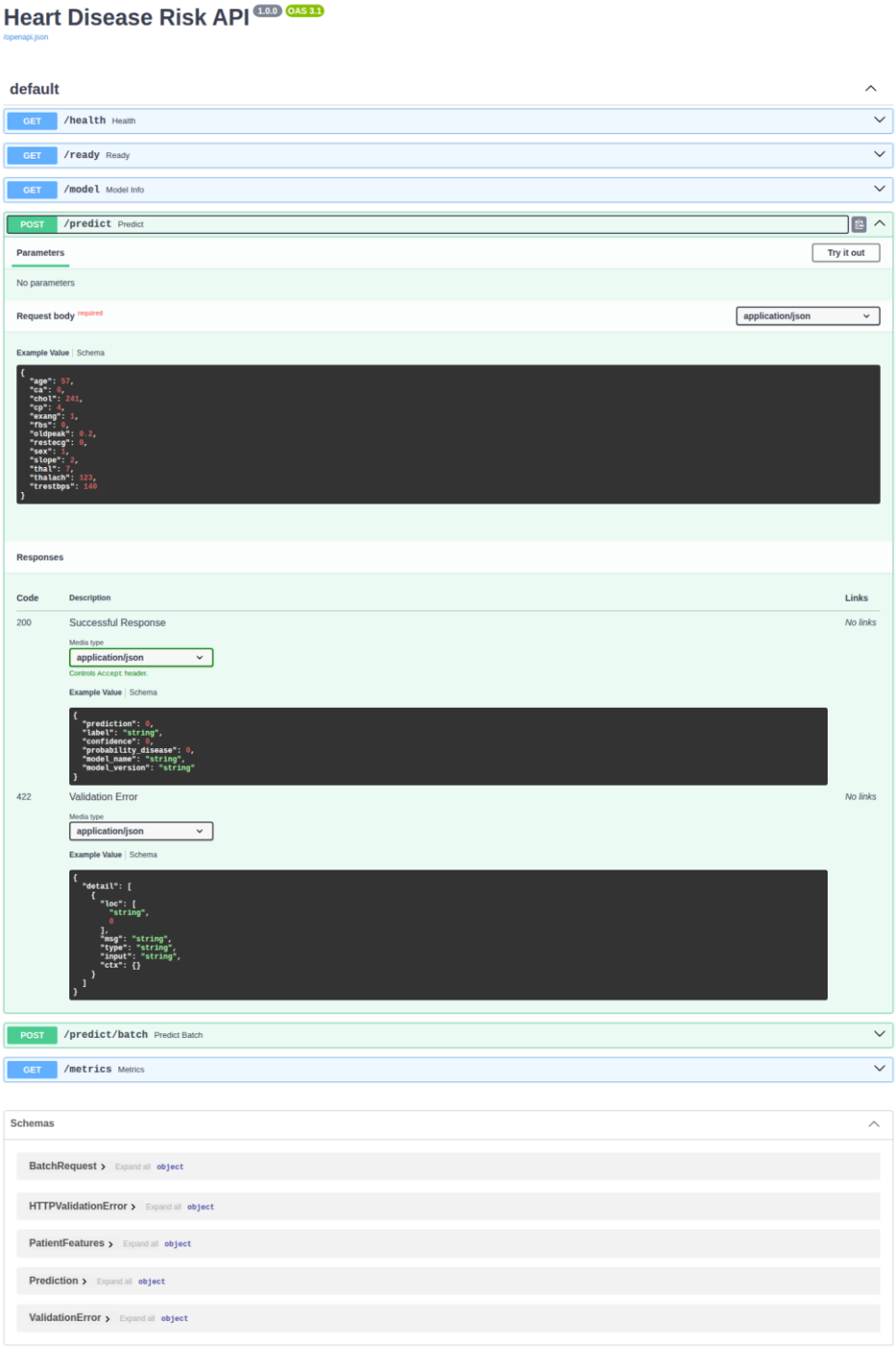


Figure 22. Auto-generated OpenAPI docs for the deployed API, POST /predict expanded